

Customers Table

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    email VARCHAR(50),  
    city VARCHAR(50)  
);
```

```
INSERT INTO Customers (customer_id, name, email, city) VALUES  
(1, 'Alice', 'alice@mail.com', 'Delhi'),  
(2, 'Bob', 'bob@mail.com', 'Mumbai'),  
(3, 'Charlie', 'charlie@mail.com', 'Pune');
```

Orders Table

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE,  
    total_amount DECIMAL(10,2),  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
);
```

```
INSERT INTO Orders (order_id, customer_id, order_date, total_amount) VALUES  
(101, 1, '2025-09-01', 5000),  
(102, 2, '2025-09-02', 7000),  
(103, 1, '2025-09-05', 3000);
```

Products Table

```
CREATE TABLE Products (  
    product_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    price DECIMAL(10,2)  
);
```

```
INSERT INTO Products (product_id, name, price) VALUES  
  
(201, 'Laptop', 40000),  
  
(202, 'Mouse', 500),  
  
(203, 'Keyboard', 1000);
```

Order_Items Table

```
CREATE TABLE Order_Items (  
    order_item_id INT PRIMARY KEY,  
    order_id INT,  
    product_id INT,  
    quantity INT,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);
```

```
INSERT INTO Order_Items (order_item_id, order_id, product_id, quantity) VALUES  
  
(1, 101, 201, 1),  
  
(2, 101, 202, 2),  
  
(3, 102, 203, 5),  
  
(4, 103, 202, 1);
```

1. Basic SELECT Query

-- List all customers

```
SELECT * FROM Customers;
```

Output:

customer_id	name	email	city
1	Alice	alice@mail.com	Delhi
2	Bob	bob@mail.com	Mumbai
3	Charlie	charlie@mail.com	Pune

2. SELECT with WHERE and ORDER BY

-- Orders where total_amount > 4000, sorted by total_amount descending

```
SELECT *
```

```
FROM Orders
```

```
WHERE total_amount > 4000
```

```
ORDER BY total_amount DESC;
```

Output:

order_id	customer_id	order_date	total_amount
102	2	2025-09-02	7000
101	1	2025-09-01	5000

3. Aggregate Functions and GROUP BY

-- Total amount spent by each customer

```
SELECT customer_id, SUM(total_amount) AS total_spent
```

```
FROM Orders
```

```
GROUP BY customer_id;
```

Output:

customer_id	total_spent
-------------	-------------

1	8000
---	------

2	7000
---	------

4. INNER JOIN Example

-- Order details with customer names

```
SELECT o.order_id, c.name AS customer_name, o.total_amount
```

```
FROM Orders o
```

```
INNER JOIN Customers c ON o.customer_id = c.customer_id;
```

Output:

order_id	customer_name	total_amount
----------	---------------	--------------

101	Alice	5000
-----	-------	------

102	Bob	7000
-----	-----	------

103	Alice	3000
-----	-------	------

5. Multiple JOINS

-- Full order info with products

```
SELECT o.order_id, c.name AS customer_name, p.name AS product_name, oi.quantity, p.price
```

```
FROM Orders o
```

```
INNER JOIN Customers c ON o.customer_id = c.customer_id
```

```
INNER JOIN Order_Items oi ON o.order_id = oi.order_id
```

```
INNER JOIN Products p ON oi.product_id = p.product_id;
```

Output:

order_id	customer_name	product_name	quantity	price
----------	---------------	--------------	----------	-------

101	Alice	Laptop	1	40000
-----	-------	--------	---	-------

order_id	customer_name	product_name	quantity	price
----------	---------------	--------------	----------	-------

101	Alice	Mouse	2	500
102	Bob	Keyboard	5	1000
103	Alice	Mouse	1	500

6. Subquery Example

-- Customers who have spent more than 7000

SELECT name

FROM Customers

WHERE customer_id IN (

 SELECT customer_id

 FROM Orders

 GROUP BY customer_id

 HAVING SUM(total_amount) > 7000

);

Output:

name

Alice

7. Create a View

-- View for total spending per customer

CREATE VIEW Customer_Spending AS

SELECT c.customer_id, c.name, SUM(o.total_amount) AS total_spent

FROM Customers c

JOIN Orders o ON c.customer_id = o.customer_id

GROUP BY c.customer_id, c.name;

-- Use the view

```
SELECT * FROM Customer_Spending;
```

Output:

	customer_id	name	total_spent
--	--------------------	-------------	--------------------

1		Alice	8000
---	--	-------	------

2		Bob	7000
---	--	-----	------

8. Optimize Query with Index

-- Create an index on Orders.customer_id